

Python Sets

Sets are used to store multiple items in a single variable.

Set is one of 4 built-in data types in Python used to store collections of data, the other 3 are List, Tuple, and Dictionary, all with different qualities and usage.

A set is a collection which is *unordered*, *unchangeable**, and *unindexed*.

Note: Sets are unordered, so you cannot be sure in which order the items will appear.

A set can contain different data types:

To determine how many items a set has, use the [len\(\)](#) function.

Note: The values [False](#) and 0 are considered the same value in sets, and are treated as duplicates:

Note: The values [True](#) and 1 are considered the same value in sets, and are treated as duplicates:

Set items are unchangeable, meaning that we cannot change the items after the set has been created.

Set items can appear in a different order every time you use them, and cannot be referred to by index or key.

Set items are unordered, unchangeable, and do not allow duplicate values.

Access Items

You cannot access items in a set by referring to an index or a key.

But you can loop through the set items using a [for](#) loop, or ask if a specified value is present in a set, by using the [in](#) keyword.

Change Items

Once a set is created, you cannot change its items, but you can add new items.

To add one item to a set use the [add\(\)](#) method.

Example

Add an item to a set, using the `add()` method:

```
thisset = {"apple", "banana", "cherry"}  
  
thisset.add("orange")  
  
print(thisset)
```

Add Sets

To add items from another set into the current set, use the `update()` method.

Example

Add elements from `tropical` into `thisset`:

```
thisset = {"apple", "banana", "cherry"}  
tropical = {"pineapple", "mango", "papaya"}  
  
thisset.update(tropical)  
  
print(thisset)
```

Add Any Iterable

The object in the `update()` method does not have to be a set, it can be any iterable object (tuples, lists, dictionaries etc.).

Example

Add elements of a list to a set:

```
thisset = {"apple", "banana", "cherry"}  
mylist = ["kiwi", "orange"]  
  
thisset.update(mylist)  
  
print(thisset)
```

Remove Item

To remove an item in a set, use the `remove()`, or the `discard()` method.

Example

Remove "banana" by using the `remove()` method:

```
thisset = {"apple", "banana", "cherry"}  
  
thisset.remove("banana")  
  
print(thisset)
```

Note: If the item to remove does not exist, `remove()` will raise an error.

Example

Remove "banana" by using the `discard()` method:

```
thisset = {"apple", "banana", "cherry"}  
  
thisset.discard("banana")  
  
print(thisset)
```

Try it Yourself »

Note: If the item to remove does not exist, `discard()` will **NOT** raise an error.

Example

Remove a random item by using the `pop()` method:

```
thisset = {"apple", "banana", "cherry"}  
  
x = thisset.pop()  
  
print(x)  
  
print(thisset)
```

Example

The `clear()` method empties the set:

```
thisset = {"apple", "banana", "cherry"}  
  
thisset.clear()  
  
print(thisset)
```

[Try it Yourself »](#)

Example

The `del` keyword will delete the set completely:

```
thisset = {"apple", "banana", "cherry"}  
  
del thisset  
  
print(thisset)
```

Union

The `union()` method returns a new set with all items from both sets.

Example

Join set1 and set2 into a new set:

```
set1 = {"a", "b", "c"}
set2 = {1, 2, 3}

set3 = set1.union(set2)
print(set3)
```

[Try It Yourself »](#)

You can use the `|` operator instead of the `union()` method, and you will get the same result.

Example

Use `|` to join two sets:

```
set1 = {"a", "b", "c"}
set2 = {1, 2, 3}

set3 = set1 | set2
print(set3)
```

[Try It Yourself »](#)

Join a Set and a Tuple

The `union()` method allows you to join a set with other data types, like lists or tuples.

The result will be a set.

Example

Join a set with a tuple:

```
x = {"a", "b", "c"}
y = (1, 2, 3)

z = x.union(y)
print(z)
```

[Try it Yourself »](#)

Note: The `|` operator only allows you to join sets with sets, and not with other data types like you can with the `union()` method.

Update

The `update()` method inserts all items from one set into another.

The `update()` changes the original set, and does not return a new set.

Example

The `update()` method inserts the items in set2 into set1:

```
set1 = {"a", "b", "c"}
set2 = {1, 2, 3}

set1.update(set2)
print(set1)
```

[Try It Yourself »](#)

Note: Both `union()` and `update()` will exclude any duplicate items.

You can use the `&` operator instead of the `intersection()` method, and you will get the same result.

Example

Use `&` to join two sets:

```
set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}

set3 = set1 & set2
print(set3)
```

Python frozenset

`frozenset` is an immutable version of a set.

Like sets, it contains unique, unordered, unchangeable elements.

Unlike sets, elements cannot be added or removed from a frozenset.

Creating a frozenset

Use the `frozenset()` constructor to create a frozenset from any iterable.

Example

Create a `frozenset` and check its type:

```
x = frozenset({"apple", "banana", "cherry"})
print(x)
print(type(x))
```

Method	Shortcut	Description
<u>add()</u>		Adds an element to the set
<u>clear()</u>		Removes all the elements from the set
<u>copy()</u>		Returns a copy of the set
<u>difference()</u>	=	Returns a set containing the difference between two or more sets
<u>difference_update()</u>	<u>-=</u>	Removes the items in this set that are also included in another, specified set
<u>discard()</u>		Remove the specified item
<u>intersection()</u>	<u>&</u>	Returns a set, that is the intersection of two other sets
<u>intersection_update()</u>	<u>&=</u>	Removes the items in this set that are not present in other, specified set(s)
<u>isdisjoint()</u>		Returns True if NO items of this set is present in another set

<u>issubset()</u>	$\leq$	Returns True if all items of this set is present in another set
	$\leq$	Returns True if all items of this set is present in another, <i>larger</i> set
<u>issuperset()</u>	$\geq$	Returns True if all items of another set is present in this set
	$\geq$	Returns True if all items of another, <i>smaller</i> set is present in this set
<u>pop()</u>		Removes an element from the set
<u>remove()</u>		Removes the specified element
<u>symmetric_difference()</u>	$\wedge$	Returns a set with the symmetric differences of two sets
<u>symmetric_difference_update()</u>	$\wedge$	Inserts the symmetric differences from this set and another
<u>union()</u>	$\downarrow$	Return a set containing the union of sets
<u>update()</u>	$\downarrow$	Update the set with the union of this set and others